

Python for Agentic AI — First Principles Notes

The six Python concepts you need before touching LangChain, LangGraph, or any agent framework. Each concept builds on the last. By the end, you can read any agentic AI codebase without hitting walls.

Table of Contents

1. [Decorators](#)
 2. [Dataclasses](#)
 3. [Typing Basics](#)
 4. [Enum](#)
 5. `@property` and `__call__`
 6. [Pydantic v2](#)
 7. [Interview Questions](#)
-

Concept 1 — Decorators

Design question: How do you add behavior to a function without modifying it?

Why This Exists

Every agentic AI framework is built on decorators. `@tool`, `@agent`, `@retry`, `@cached`, `@field_validator` — these aren't magic annotations. They're functions that wrap other functions. If you don't understand decorators, you can't read framework source code, you can't debug when things go wrong, and you can't write your own.

Functions Are First-Class Objects

Python lets you assign functions to variables, pass them as arguments, and return them from other functions:

```
def greet():
    print("Hello")

x = greet    # assign the function object (not calling it)
x()          # Hello
```

This is the foundation. A decorator is just a function that receives a function and returns a new one.

Building a Decorator Manually

```
def my_decorator(func):
    def wrapper():
        print("Before")
        func()          # call the original
        print("After")
    return wrapper      # return the wrapper — don't call it

def greet():
    print("Hello")

greet = my_decorator(greet)  # greet now points to wrapper
greet()                     # Before / Hello / After
```

Why return a function? If the decorator called `func()` directly and returned `None`, then `greet` would become `None` after decoration. The wrapper delays execution until someone actually calls `greet()`.

@ Is Just Syntactic Sugar

```
@my_decorator
def greet():
    print("Hello")

# identical to:
greet = my_decorator(greet)
```

That's it. `@` is not a language feature with special powers. It's a one-line shorthand for reassignment.

Handling Any Function Signature

A real decorator must work with any function — different arguments, return values, everything:

```

from functools import wraps

def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        import time
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.perf_counter()-start:.4f}s")
        return result
    return wrapper

@timer
def train(epochs, lr=0.001):
    return f"Trained for {epochs} epochs"

```

`*args` and `**kwargs` forward any arguments. `result = func(...)` captures and returns whatever the original function returned.

Why `functools.wraps` Matters

Without it:

```
print(greet.__name__) # "wrapper" - the original name is lost
```

This breaks debugging, stack traces, `help()`, and any framework that inspects function metadata (which is most of them). `@wraps(func)` copies `__name__`, `__doc__`, `__module__`, and `__annotations__` from the original function to the wrapper.

Decorator Factory — When the Decorator Takes Arguments

```

@logger("DEBUG")          # not @logger - notice the parentheses and argument
def search(query):
    ...

```

This requires one extra nesting level:

```

def logger(level):          # factory - takes the argument
    def decorator(func):    # actual decorator - takes the function
        @wraps(func)
        def wrapper(*args, **kwargs):
            print(f"[{level}] {func.__name__}")
            return func(*args, **kwargs)
        return wrapper
    return decorator

```

Python evaluates `@logger("DEBUG")` as: call `logger("DEBUG")` first → get back a decorator → apply that decorator to the function. So it's `func = logger("DEBUG")(func)`.

Where You See This in Agentic AI

Decorator	Framework	What it does
<code>@tool</code>	LangChain	Wraps a function as an agent tool
<code>@retry</code>	Tenacity	Retries failed LLM API calls
<code>@lru_cache</code>	stdlib	Caches expensive embeddings
<code>@field_validator</code>	Pydantic	Validates a model field
<code>@app.get("/")</code>	FastAPI	Registers a route handler
<code>@abstractmethod</code>	abc	Enforces a contract in base classes

Every single one follows the same pattern: function in, wrapper out.

One-Liner

A decorator takes a function, wraps it, and returns a new function. `@` is shorthand for `func = decorator(func)`.
Always use `@wraps` to preserve metadata.

Concept 2 — Dataclasses

Design question: How do you stop writing the same `__init__`, `__repr__`, `__eq__` boilerplate across dozens of data-holding classes?

Why This Exists

AI systems are full of data containers — configs, messages, tool results, embeddings metadata, API responses. Without `@dataclass`, every one of them needs 15+ lines of boilerplate for `__init__`, `__repr__`, and `__eq__`. With it, you declare fields and Python generates everything.

Before and After

Without:

```
class ToolResult:
    def __init__(self, tool_name, output, success):
        self.tool_name = tool_name
        self.output = output
        self.success = success

    def __repr__(self):
        return f"ToolResult(tool_name={self.tool_name!r}, output={self.output!r}, success={self.success!r})"

    def __eq__(self, other):
        return (self.tool_name == other.tool_name and
                self.output == other.output and
                self.success == other.success)
```

With:

```
from dataclasses import dataclass

@dataclass
class ToolResult:
    tool_name: str
    output: str
    success: bool
```

Same behavior. Three lines.

What Gets Generated

Method	What it does
<code>__init__</code>	<code>ToolResult("search", "Found 10", True)</code> works
<code>__repr__</code>	<code>print(r)</code> → <code>ToolResult(tool_name='search', ...)</code>
<code>__eq__</code>	Compares by field values, not object identity

The Mutable Default Trap

```
@dataclass
class Agent:
    tools: list = [] # ERROR — mutable default shared across all instances
```

Python prevents this. The fix:

```
from dataclasses import field

@dataclass
class Agent:
    tools: list = field(default_factory=list)  # each instance gets its own list
```

`default_factory=list` calls `list()` separately for every new object.

`field()` Options

Option	Effect	Example
<code>default_factory</code>	Fresh mutable default per instance	<code>field(default_factory=list)</code>
<code>repr=False</code>	Hide from <code>print()</code> output	<code>password: str = field(repr=False)</code>
<code>compare=False</code>	Exclude from <code>==</code>	<code>id: int = field(compare=False)</code>
<code>init=False</code>	Not in constructor, set in <code>__post_init__</code>	<code>score: float = field(init=False)</code>

`__post_init__` — Computed Fields

```
@dataclass
class Rectangle:
    width: float
    height: float
    area: float = field(init=False)

    def __post_init__(self):
        self.area = self.width * self.height
```

`frozen=True` — Immutable Configs

```
@dataclass(frozen=True)
class AgentConfig:
    model: str
    temperature: float
    max_tokens: int

cfg = AgentConfig("gpt-5", 0.7, 1000)
cfg.model = "gpt-4"  # FrozenInstanceError — blocked
```

Frozen dataclasses are also **hashable** — usable as dict keys and in sets.

Limitation: `frozen=True` is shallow. A `list` inside a frozen dataclass is still mutable. Use `tuple` for truly immutable collections.

One-Liner

`@dataclass` generates `__init__`, `__repr__`, `__eq__` from field declarations. Use `field(default_factory=...)` for mutable defaults, `frozen=True` for immutable configs.

Concept 3 — Typing Basics

Design question: How do you tell Python, your editor, and Pydantic what type a variable should be?

Why This Exists

Type hints are the language Pydantic speaks. Without them, Pydantic can't build validators, FastAPI can't generate API docs, and your editor can't autocomplete. They also make code readable — when you see `tools: list[str]`, you know what's inside without reading the implementation.

Modern Syntax (Python 3.10+)

Type	Meaning	Syntax
Optional	May be None	<code>str None</code>
Union	One of several types	<code>int str</code>
List	Typed list	<code>list[str]</code>
Dict	Typed dict	<code>dict[str, float]</code>
Tuple	Fixed-size	<code>tuple[int, int]</code>
Any	Anything (escape hatch)	<code>Any</code>
Callable	Function signature	<code>Callable[[str], bool]</code>
Class itself	Not an instance	<code>type[BaseAgent]</code>
Fixed values	Constants only	<code>Literal["fast", "slow"]</code>

Rule: only `Any`, `Callable`, and `Literal` still need `from typing import`. Everything else uses built-in syntax.

What You'll Actually Write in Agentic AI

```
api_key: str | None = None           # optional config
tools: list[str]                     # tool names
metadata: dict[str, Any]              # JSON payloads
callback: Callable[[str], None]      # event hooks
mode: Literal["chat", "tool", "planner"] # fixed config values
agent_cls: type[BaseAgent]           # passing classes
```

The Key Insight

Type hints **do nothing at runtime** in plain Python. `add("hello", "world")` works even if `add` is annotated as `(a: int, b: int) -> int`. The annotations are metadata for editors, linters, and Pydantic. Pydantic is the thing that actually reads them and enforces them.

One-Liner

Type hints are the schema language for Pydantic, FastAPI, and your editor. They don't enforce anything alone — but everything in modern AI tooling is built on them.

Concept 4 — Enum

Design question: *How do you define a fixed set of allowed values that can't be silently mistyped?*

Why This Exists

Agent states, tool types, LLM providers, priority levels — all of these are a fixed set of values. Using plain strings means typos compile and fail silently at runtime:

```
run_agent("Chat")      # typo — no error, wrong behavior
run_agent("chatt")     # typo — no error, silent failure
```

Basic Enum

```
from enum import Enum

class AgentState(Enum):
    IDLE = "idle"
    RUNNING = "running"
    FINISHED = "finished"
    ERROR = "error"

state = AgentState.RUNNING
print(state.name)      # RUNNING
print(state.value)     # running
```

Now `AgentState("unknown")` raises `ValueError` — runtime safety.

Variants

`IntEnum` — members behave like integers:

```
from enum import IntEnum

class Priority(IntEnum):
    LOW = 1
    MEDIUM = 2
    HIGH = 3

Priority.HIGH > Priority.LOW  # True
```

`auto()` — when actual values don't matter:

```
from enum import auto

class Status(Enum):
    PENDING = auto()  # 1
    RUNNING = auto()  # 2
    DONE = auto()     # 3
```

Enum vs Literal

Need	Use
Shared across modules, needs iteration, runtime validation	<code>Enum</code>
Type hint in one function, no runtime check needed	<code>Literal</code>


```
# Enum - central definition, shared everywhere
class Provider(Enum):
    OPENAI = "openai"
    ANTHROPIC = "anthropic"

# Literal - one-off type hint
def set_mode(mode: Literal["fast", "slow"]) -> None: ...
```

One-Liner

Enum gives you named, safe, autocomplete-friendly constants with runtime validation. Use it for anything that has a fixed set of values shared across your codebase.

Concept 5 — @property and __call__

Design question: How do you add validation to attributes without breaking callers? And how do you make an object behave like a function?

@property — Methods That Look Like Attributes

The problem: you want to validate `age` on assignment, but changing `u.age = 25` to `u.set_age(25)` breaks every caller.

```
class User:
    def __init__(self, age: int):
        self._age = age

    @property
    def age(self) -> int:
        return self._age

    @age.setter
    def age(self, value: int):
        if value < 0:
            raise ValueError("Age cannot be negative")
        self._age = value
```

Now `u.age = -5` raises `ValueError`, but the syntax hasn't changed — callers still write `u.age`, not `u.get_age()`.

Computed properties:

```
class AgentConfig:
    def __init__(self, model, max_tokens):
        self.model = model
        self.max_tokens = max_tokens

    @property
    def summary(self):
        return f"{self.model} (max_tokens={self.max_tokens})"

print(cfg.summary)  # no parentheses — looks like an attribute
```

How it works: Python uses the descriptor protocol. When it sees `u.age` and finds a property descriptor on the class, it calls `User.age.__get__(u, User)` instead of directly reading the attribute. You don't need to memorize this — just know that methods are running behind attribute syntax.

__call__ — Objects That Behave Like Functions

```
class SimilarityScorer:
    def __init__(self, threshold: float):
        self.threshold = threshold

    def __call__(self, score: float) -> bool:
        return score >= self.threshold

scorer = SimilarityScorer(0.8)
scorer(0.92)  # True — called like a function
scorer(0.65)  # False
```

Why not just use a function? Functions can't carry state. A scorer needs its threshold, a tool needs its API client, a tokenizer needs its vocabulary. `__call__` gives you function syntax with object internals.

The rule is simple: stateless → regular function. Stateful → `__call__`.

This pattern appears everywhere: PyTorch `nn.Module`, sklearn transformers, LangChain tools, FastAPI dependencies.

One-Liner

`@property` hides methods behind attribute syntax for validation and computed values. `__call__` makes objects callable for when functions need to carry state.

Concept 6 — Pydantic v2

Design question: *How do you safely convert untrusted external data into validated Python objects?*

Why This Is the Most Important Concept Here

Every agentic AI system receives data from outside — JSON from LLMs, HTTP requests, tool outputs, config files, environment variables. None of it can be trusted. Pydantic is the boundary between the messy outside world and your clean Python objects. If you learn one thing from this entire curriculum, it should be this.

Dataclass Trusts You. Pydantic Doesn't.

```
# Dataclass — stores "0.8" as a string without complaint
@dataclass
class Config:
    temperature: float

cfg = Config("0.8")
print(type(cfg.temperature))  # str — silent bug

# Pydantic — converts "0.8" to 0.8, rejects "high"
class Config(BaseModel):
    temperature: float

cfg = Config(temperature="0.8")
print(type(cfg.temperature))  # float — validated and converted
```

The Validation Pipeline

Every `BaseModel` follows this pipeline on construction:

```
Incoming data
↓
Read field annotations (type hints)
↓
Convert compatible types ("0.8" → 0.8)
↓
Apply Field() constraints (ge=0, le=2)
↓
Run @field_validator (single-field business rules)
↓
Run @model_validator (cross-field rules)
↓
Validated Python object
```

`Field()` — Constraints Beyond Types

Type hints answer "what type is this?" `Field()` answers "what rules apply?"

```
from pydantic import BaseModel, Field

class AgentConfig(BaseModel):
    model: str = Field(default="gpt-4", min_length=3)
    temperature: float = Field(ge=0, le=2, description="Sampling temperature")
    max_tokens: int = Field(gt=0, le=4096)
```

Constraint	Meaning
gt / ge	Greater than / greater than or equal
lt / le	Less than / less than or equal
min_length / max_length	String or list length
pattern	Regex for strings
alias	Map from external key name (e.g. camelCase JSON)
description	Used by FastAPI for docs, by LLMs for tool schemas

Aliases — when APIs use camelCase:

```
class ToolCall(BaseModel):
    tool_name: str = Field(alias="toolName")

ToolCall(toolName="search")  # maps to tool_name internally
```

Nested Models and Lists of Models

Real data is hierarchical. An agent has tools. A workflow has agents.

```
class Tool(BaseModel):
    name: str
    description: str

class Agent(BaseModel):
    name: str
    tools: list[Tool]  # list of nested models

# Pass dicts — Pydantic converts each one to a Tool
agent = Agent(
    name="ResearchBot",
    tools=[
        {"name": "search", "description": "Search the web"},
        {"name": "calculator", "description": "Solve math"},
    ]
)

print(agent.tools[0].name)  # search
print(type(agent.tools[0]))  # Tool — not dict
```

Validation is recursive. If a tool dict has invalid fields, the entire agent construction fails with a clear error pointing to the exact nested field.

Serialization: `model_dump()` and `model_dump_json()`

```
agent.model_dump()  # → dict (for internal use)
agent.model_dump_json()  # → JSON string (for APIs)
```

Nested models serialize recursively.

Parsing: `model_validate()` and `model_validate_json()`

```
# From dict
config = AgentConfig.model_validate({"model": "gpt-5", "temperature": "0.7", "max_tokens": "1000"})

# From JSON string — no json.loads() needed
config = AgentConfig.model_validate_json('{"model": "gpt-5", "temperature": 0.7, "max_tokens": 1000}')
```

The complete lifecycle in an agentic system:

```
LLM JSON output → model_validate_json() → Validated object
                                     ↓
                                     Agent logic
                                     ↓
Next tool call ← model_dump_json() ← Modified object
```

@field_validator — Single Field Rules

When type constraints aren't enough:

```
from pydantic import field_validator

class Tool(BaseModel):
    name: str = Field(min_length=3)
    max_retries: int = Field(ge=1, le=5)

    @field_validator("name")
    @classmethod
    def uppercase_name(cls, value: str) -> str:
        return value.upper() # transform before storing

Tool(name="search", max_retries=3).name # "SEARCH"
```

Field validators run **after** type validation. They receive the already-converted value and can transform or reject it.

@model_validator — Cross-Field Rules

When validation depends on relationships between fields:

```
from pydantic import model_validator

class LLMConfig(BaseModel):
    model: str
    temperature: float

    @model_validator(mode="after")
    def validate_o1_temperature(self):
        if self.model == "o1" and self.temperature != 1.0:
            raise ValueError("o1 only supports temperature=1.0")
        return self
```

`mode="after"` means all fields are already validated individually — `self` has clean data. This is where business logic goes.

Complete Example — Everything Together

```

from pydantic import BaseModel, Field, field_validator, model_validator

class Tool(BaseModel):
    name: str = Field(min_length=2)
    description: str
    enabled: bool = True

class AgentConfig(BaseModel):
    model: str = Field(default="gpt-4")
    temperature: float = Field(ge=0, le=2)
    tools: list[Tool]

    @field_validator("model")
    @classmethod
    def validate_model(cls, value):
        allowed = {"gpt-4", "gpt-5", "claude"}
        if value not in allowed:
            raise ValueError(f"Unsupported model: {value}")
        return value

    @model_validator(mode="after")
    def require_active_tools(self):
        if not any(t.enabled for t in self.tools):
            raise ValueError("Need at least one enabled tool")
        return self

# Load from JSON → validate → use → serialize back
config = AgentConfig.model_validate({
    "model": "gpt-5",
    "temperature": "0.7",
    "tools": [{"name": "search", "description": "Search the web"}]
})
print(config.model_dump_json(indent=2))

```

One-Liner

Pydantic is the boundary between untrusted external data and clean Python objects. Type hints become a schema, `Field()` adds constraints, `@field_validator` adds single-field rules, `@model_validator` adds cross-field rules. `model_validate()` parses in, `model_dump()` serializes out.

Master Summary

Concept	Core Idea	When You See It
Decorator	Function that wraps a function	<code>@tool</code> , <code>@retry</code> , <code>@field_validator</code>
<code>@wraps</code>	Preserve original function's metadata	Every well-written decorator
Decorator factory	Decorator that takes arguments	<code>@logger("DEBUG")</code> , <code>@app.get("/")</code>
Dataclass	Auto <code>__init__</code> + <code>__repr__</code> + <code>__eq__</code>	Configs, messages, results
<code>field()</code>	Mutable defaults, repr/compare control	<code>field(default_factory=list)</code>
<code>frozen=True</code>	Immutable + hashable	Agent configs, coordinates
Typing	Schema language for Pydantic and editors	<code>str None</code> , <code>list[str]</code> , <code>Callable</code>
Enum	Named set of safe constants	Agent states, tool types, providers
<code>@property</code>	Method behind attribute syntax	Computed/validated attributes
<code>__call__</code>	Object callable like a function	Scorers, tools, tokenizers, models
BaseModel	Type hints → runtime validation schema	Every Pydantic model
<code>Field()</code>	Constraints + metadata beyond types	<code>ge=0</code> , <code>alias</code> , <code>description</code>
Nested models	Recursive validation	Agent → Tools → Parameters
<code>model_validate()</code>	External data → validated object	Parsing LLM output, API responses
<code>model_dump()</code>	Validated object → dict/JSON	Sending to APIs, logging
<code>@field_validator</code>	Custom single-field rule	Transform names, check allowed values
<code>@model_validator</code>	Cross-field business rule	"o1 requires temperature=1.0"

Interview Questions

Decorators

Q1. What does `@decorator` actually do? Rewrite it without the `@` syntax.

`@decorator` over a function `f` is equivalent to `f = decorator(f)`. Python passes the function to the decorator, which returns a new function (the wrapper), and reassigns the name to point to the wrapper.

Q2. What breaks if you don't use `functools.wraps`, and why does it matter in production?

The wrapper replaces the original function's `__name__`, `__doc__`, and `__module__`. This breaks debugging (stack traces show "wrapper"), `help()` shows wrong docs, and frameworks that inspect function metadata (FastAPI, Flask, pytest) may behave incorrectly.

Q3. What is a decorator factory? Write one that takes a `level` argument for logging.

A decorator factory is a function that returns a decorator. It adds one extra nesting level: the factory takes the argument, returns the actual decorator, which takes the function and returns the wrapper. `@logger("INFO")` evaluates as `func = logger("INFO")(func)`.

Dataclasses

Q4. What methods does `@dataclass` auto-generate?

`__init__`, `__repr__`, and `__eq__`. Optionally `__hash__` (with `frozen=True`) and `__lt__`/`__gt__` etc. (with `order=True`).

Q5. Why can't you write `tools: list = []` in a dataclass, and what's the fix?

Mutable defaults would be shared across all instances (same object). Python raises an error to prevent this. Fix: `tools: list = field(default_factory=list)` — calls `list()` separately per instance.

Q6. What does `frozen=True` enable beyond immutability?

Hashing. Frozen dataclasses get an auto-generated `__hash__`, so they can be used as dict keys and in sets. This is useful for caching configs or deduplicating results.

Typing

Q7. Do type hints enforce anything at runtime in plain Python?

No. `add("hello", "world")` works even with `def add(a: int, b: int)`. Type hints are metadata for editors, linters (mypy/pyright), and validation frameworks like Pydantic. Pydantic is what makes them runtime-enforceable.

Q8. What's the modern syntax for `Optional[str]`, `Union[int, str]`, and `List[str]`?

`str | None`, `int | str`, `list[str]`. Available from Python 3.10+. Only `Any`, `Callable`, and `Literal` still require `from typing import`.

Enum

Q9. When would you use `Enum` vs `Literal`?

`Enum` when the values are shared across modules, you need iteration, `.name` / `.value`, or runtime validation (e.g., `AgentState("running")` raises on invalid input). `Literal` when it's just a type hint used in one place and you don't need any of those features.

Q10. What is `IntEnum` and when do you use it?

`IntEnum` members behave like integers — they support comparison operators (`>`, `<`) and can be used wherever an `int` is expected. Use it for priority levels, status codes from APIs, or anything that maps to numeric values.

`@property` and `__call__`

Q11. What problem does `@property` solve that you can't solve with plain attributes?

Validation on assignment and computed values without changing the public API. `u.age = -5` can trigger a setter that raises `ValueError`, while callers still write `u.age` instead of `u.set_age()`. Also enables derived attributes like `rectangle.area` that compute on access.

Q12. When should you use `__call__` instead of a regular function?

When the callable needs persistent state — API clients, thresholds, caches, model configs, rate limiters. A regular function can't carry these without globals or closures. `__call__` gives you function syntax (`tool(query)`) backed by an object that stores configuration internally.

Pydantic

Q13. What is the fundamental difference between `@dataclass` and `BaseModel`?

A `dataclass` trusts the input and stores it as-is. A `BaseModel` validates, coerces types (e.g., `"0.8" → 0.8`), and rejects invalid data with detailed errors. `Dataclass` is a container. `BaseModel` is a container + validator.

Q14. What does `Field()` add that type hints alone can't express?

Numeric constraints (`ge`, `le`, `gt`, `lt`), string constraints (`min_length`, `pattern`), defaults, aliases (for camelCase JSON), descriptions (for API docs and tool schemas), and examples. These are metadata about the field, not its type.

Q15. What is the difference between `@field_validator` and `@model_validator`?

`@field_validator` validates one field in isolation — runs after type coercion, receives the converted value, can transform or reject it. `@model_validator(mode="after")` runs after all fields are validated — has access to the fully constructed `self`, used for cross-field business rules (e.g., "o1 requires temperature=1.0").

Q16. Explain the full data lifecycle in an agentic system using Pydantic.

External data (LLM JSON, API response) enters via `model_validate()` or `model_validate_json()` → Pydantic validates, coerces types, runs field and model validators → produces a trusted Python object → agent logic processes it → `model_dump()` or `model_dump_json()` serializes it back to dict/JSON → sent to the next tool, API, or database. Pydantic handles both directions.

Q17. You receive this JSON from an LLM. Write a Pydantic model that validates it:

```
{"tool": "search", "query": "Python decorators", "max_results": "5"}
```

```
python class ToolCall(BaseModel): tool: str query: str = Field(min_length=1) max_results: int = Field(ge=1,
```

`le=20)` `"5"` gets coerced to `int`. `max_results` is bounded. `query` must be non-empty. Invalid data raises `ValidationError` with field-level messages.

Q18. What happens if you nest one `BaseModel` inside another and pass a dict for the nested field?

Pydantic automatically calls the nested model's constructor on the dict, validating it recursively. `Agent(tool={ "name": "search" })` creates a `Tool(name="search")` inside the agent. If the dict has invalid fields, the error message points to the exact nested field that failed.